OPEN SOURCE & SYSTEMS ENGINEERING

POLYGLOT SWE

Java • Python • C++ • TypeScript • Algorithmic Optimization • Complex Codebase Refactoring

Focus: Open-Source Development & AI Technical Benchmarks
Availability: Immediate Onboarding (Within 24–48 Hours)
Core Languages: Java 21, Python 3.x, C++, TypeScript, C#
Model: Senior Software Engineering Team

High-Velocity Open Source Engineering, Bug Triage & Algorithmic Optimization

A senior, battle-tested software engineering team proficient in reading, debugging, and refactoring large-scale open-source codebases. Combining deep polyglot fluency across Java, Python, C++, and TypeScript with rigorous algorithmic problem-solving, automated test coverage, and clear technical documentation to solve complex software engineering challenges.

Languages: **Java 21 • Python 3.12 • C++ • TypeScript • C#**
Workflows: **Git • GitHub PRs • Issue Triage • SWE-bench**
Discipline: **Clean Code • Algorithm Profiling • Unit Testing**
Execution: **Reproducible Bug Fixes • High-Performance Refactoring**
Onboarding: **Ready within 24–48 Hours of Selection**

1. ENGINEERING CAPABILITIES & CORE STRENGTHS

Mastering Unfamiliar Repositories Quickly and Delivering Production-Grade Code

Navigating & Mastering Existing Codebases

CODE MASTERY

In advanced open-source and AI benchmark environments, the primary hurdle is rapidly understanding architectural patterns, dependency graphs, and implicit assumptions across massive repos:

- **Fast Architectural Deconstruction:** Quickly parsing call graphs, module boundaries, and concurrency patterns in multi-thousand-line repositories.
- **Root-Cause Debugging:** Isolating edge-case defects, memory leaks, race conditions, and numerical inaccuracies using systematic reproduction harnesses.
- **Minimal-Diff Refactoring:** Writing surgical, idiomatic code modifications that solve the issue cleanly without introducing regressions or side-effects.

Algorithmic Rigor & Clean Code Standards

ENGINEERING DEPTH

We treat code quality as an uncompromising foundation across all programming languages:

- **Complexity Optimization:** Analyzing Big-O time and space complexity, replacing $O(n^2)$ bottlenecks with O(n log n) or O(1) structures.
- **Defensive Testing & Verification:** Backing every feature and bug fix with thorough unit tests (JUnit, pytest, Catch2, Jest) and edge-case assertions.
- **Exemplary Documentation:** Writing clear, human-readable PR descriptions, documenting architectural trade-offs, and providing reproducible verification steps.

2. TECHNICAL PROFICIENCY & LANGUAGE MATRIX

Production Depth Across the Entire Required Tech Stack

LANGUAGE & STACK	CORE COMPETENCIES & ENGINEERING APPLICATIONS	VERIFIED PRECEDENTS
Java (8, 17, 21)	Object-oriented design, multi-threading, concurrency utilities, Spring Boot microservices, memory optimization, JVM garbage collection tuning, and JUnit testing.	DIMO Gate Pass enterprise microservices, Kafka event streaming, and SINGER financial platforms.
Python (3.8–3.12)	Data structures, async I/O, algorithmic problem-solving, FastAPI/Flask services, data pipelines, pytest suites, and deterministic mathematical calculations.	*InvoiceIQ* automated reconciliation engine with SHA-256 idempotency and data parsing pipelines.
C / C++	Pointer arithmetic, manual memory management (RAII), STL containers and algorithms, low-level data structures, profiling, and performance-critical systems.	Academic systems programming, competitive programming problem solving, and algorithm implementation.
TypeScript / JavaScript	Modern ES6+, asynchronous event loops, Node.js backend services, typed domain modeling, React/Next.js architectures, and Jest/Mocha unit test suites.	Adrik Kuwait bilingual BFF microservices, Al Ain Museum V2 frontend, and enterprise dashboard platforms.
Git & Open Source Tooling	Advanced git branching, rebasing, interactive conflict resolution, bisecting regression commits, GitHub Actions CI/CD pipelines, and pull request reviews.	Active GitHub contributions, multi-environment CI/CD deployment pipelines, and code review governance.

Immediate Availability: Ready to Begin Within 24–48 Hours
We understand this project requires immediate engagement. Our engineers are available to begin onboarding, repository access setup, and technical screening/assessment tasks within 24 to 48 hours of acceptance.

ENGINEERING METHODOLOGY & PROBLEM SOLVING

Systematic Debugging, Algorithmic Optimization, and Codebase Refactoring

METHODOLOGY

Approach: Test-Driven & Root-Cause Analysis
Quality Gates: SonarQube, JaCoCo, Pytest
Git Etiquette: Atomic Commits & Clear PR Walkthroughs

3. SYSTEMATIC PROBLEM-SOLVING & BUG TRIAGE LIFECYCLE

How We Approach Complex Bugs and Feature Implementations in Open Source

1. Minimal Reproduction Harness — STEP 1

Before changing a single line of code, we isolate the reported bug by writing a failing unit test or minimal reproduction script. This guarantees unambiguous proof of the defect and provides an objective benchmark for verifying the eventual fix.

2. Root-Cause Analysis & Tracing — STEP 2

Using interactive debuggers, stack trace analysis, and `git bisect` when necessary, we pinpoint the exact commit, invariant violation, off-by-one condition, or type mismatch that caused the issue—ensuring we treat the underlying defect rather than symptoms.

3. Surgical, Idiomatic Refactoring — STEP 3

We implement the fix or new feature strictly following the repository's established style guides, architectural conventions, and design patterns. We maintain minimal diff footprints to facilitate rapid peer review and reduce regression surface area.

4. Verification, Edge Cases & Documentation — STEP 4

We expand test coverage to include boundary conditions (nulls, empty collections, overflow values, concurrent access). We document the technical reasoning, changes made, and verification steps in clean, structured Markdown pull requests.

4. ALGORITHMIC OPTIMIZATION & COMPLEXITY ENGINEERING

Transforming Sluggish Execution into High-Throughput Performance

STAGE 01: PROFILING

Bottleneck Identification

Isolating CPU hot-loops and memory bottlenecks using non-invasive flamegraphs, heap profilers (`py-spy`, `JProfiler`, `perf`), and I/O trace analysis.

➔

STAGE 02: REFACTORING

Algorithmic Optimization

- **Lookup Speed:** Replacing linear scans `O(N)` with `O(1)` Hash Sets & Maps.
- **Memoization:** Dynamic programming caching for repeated recursive sub-problems.
- **Memory Footprint:** Generators, primitive arrays, and object pooling.
- **Concurrency:** Non-blocking async routines (`asyncio`, `CompletableFuture`).

➔

STAGE 03: VERIFICATION

Benchmark & Testing

Running automated micro-benchmarks to quantify latency reduction, verify zero memory drift, and prove zero regressions against the unit test suite.

5. QUALITY STANDARDS, PULL REQUEST ETIQUETTE & DOCUMENTATION

Writing Code That Open-Source Maintainers Love to Merge

Atomic Git Commits — VERSION CONTROL

Every commit encapsulates a single logical change with clear, conventional commit messages (`feat:`, `fix:`, `refactor:`, `test:`) enabling easy cherry-picking and rollback.

Automated Linting & Types — CODE STANDARDS

Strict adherence to language formatters and linters (Checkstyle, Black, Flake8, ESLint, Clang-Format), ensuring zero lint warnings and complete static type safety.

Comprehensive Runbooks — DOCUMENTATION

Writing crystal-clear developer documentation, setup guides, API schemas, and architectural decision records (ADRs) that make solutions self-explanatory.

VERIFIED INTERNATIONAL ENGINEERING PRECEDENTS

TRACK RECORD

Production Systems in Java, Python, TypeScript, Microservices & Event Architecture

International Scope: Sri Lanka, Kuwait, UAE, Australia
Backend Tech: Java 21, Python, Node.js, Spring Boot, Docker
Quality Gates: SonarQube, JaCoCo, JUnit, Pytest

6. PRODUCTION SYSTEMS & COMPLEX PROBLEM SOLVING CASE STUDIES

Demonstrated Engineering Rigor on Real-World International Platforms

DIMO Enterprise Gate Pass & Logistics Management System

Client: DIMO Lanka • Lead: Viranga Lakshan

JAVA 21 / SPRING BOOT

Complex Microservices Architecture: Engineered high-concurrency microservices utilizing Java 21 and Spring Boot 3.4.5, managing API gateways, role-based approval hierarchies, and event-driven audit streaming.

- **Automated Code Quality & Test Gates:** Supported CI/CD workflows using Jenkins and GitLab pipelines enforcing automated SonarQube static code quality analysis and JaCoCo test coverage verification before artifact promotion.
- **Asynchronous Event Streaming:** Implemented Spring Kafka publishing immutable audit events across distributed services, integrating with SAP ERP, Microsoft Graph, and AWS S3 with ClamAV virus scanning.
- **Database Migrations:** Managed relational schema design and automated Flyway migrations in MySQL, ensuring zero downtime.

InvoiceIQ Reconciliation Engine & SINGER Sri Lanka PLC

Client: SINGER PLC & Open Systems • Janith Kavishka

PYTHON & JAVA SYSTEMS

Deterministic Financial Matching & Distribution Systems: Architected *InvoiceIQ*, an automated line-item reconciliation engine in Python and FastAPI performing deterministic 3-way matching with SHA-256 idempotency controls.

- **Enterprise Java & Event Messaging:** Built backend services using Java and Spring Boot with RabbitMQ for message-driven communication across distributed enterprise components.
- **Database Optimization & Migration:** Executed zero-downtime database migration to Amazon RDS (MySQL) for nationwide transaction tracking, tuning complex queries and eliminating performance bottlenecks.

Adrik Bilingual Media Platform

Client: Adrik (Kuwait) • Node.js / Kubernetes

NODE / TYPESCRIPT

Developed three Node.js/Express BFF microservices connecting web and mobile applications for a Kuwaiti media client. Deployed to Kubernetes via GitLab CI/CD with Keycloak OAuth2/OIDC authentication.

Castle Visualizer Analytics Platform

Client: Castle Group (Australia) • Lithumitha

DATA SYSTEMS

Led delivery of enterprise multi-dashboard analytics platform consolidating data across legacy business systems into real-time reporting, managing requirements and rigorous QA acceptance criteria.

SQUAD PROFILES & ONBOARDING READINESS

EXECUTION

Onboarding: Available Within 24–48 Hours
Communication: Fluent English (Written & Verbal)
GitHub: github.com/viranga-lakshan

Team Backgrounds, GitHub Contributions, Competitive Problem Solving & Immediate Availability

7. MEET THE SOFTWARE ENGINEERING SQUAD

Dedicated Problem Solvers with International Enterprise Experience

Viranga Lakshan

LEAD SOFTWARE ENGINEER

Associate Software Engineer, Arimac • 4+ Years Systems Experience

Polyglot software engineer specializing in Java 21, Spring Boot, Python, Node.js/TypeScript, C# .NET, Docker containerization, Kafka event streaming, and automated CI/CD code quality pipelines (SonarQube).

- Architected microservices for international clients across Kuwait, UAE, and Sri Lanka.
- Hands-on experience refactoring complex distributed codebases and optimizing algorithms.

Java 21 | Python | TypeScript | Docker | Git / CI/CD | Kafka

Janith Kavishka

SENIOR BACKEND & SYSTEMS ENGINEER

Programmer, DMS Electronics • Systems & Data Engineering

Expert in algorithmic design, data structures, Java, Python, Spring Boot, RabbitMQ, SQL query optimization, and building high-performance backend pipelines.

- Architected *InvoiceIQ* 3-way automated matching engine with SHA-256 idempotency.
- Built distributed transaction processing and AWS serverless distribution platforms.

Java / Spring | Python | SQL Tuning | RabbitMQ | Algorithms | AWS Lambda

Lithumitha Ravikumar

TECHNICAL DELIVERY & QA LEAD

Associate Project Manager, Arimac • Enterprise Systems Governance

Bridges technical problem solving with structured delivery, code review tracking, issue triage, and acceptance testing across multi-squad engineering environments.

- Managed 4 engineering squads delivering enterprise multi-tenant digital SaaS platforms.
- Coordinates technical documentation, PR reviews, and QA validation.

Issue Triage | PR Review | QA Testing | Documentation | Agile / Git

8. IMMEDIATE ONBOARDING PROTOCOL & STANDARDS

Zero Ramp-Up Delay — Delivering Value from Day One

24–48h Onboarding

AVAILABILITY

Ready to clone repositories, configure local development sandboxes, review contribution guidelines, and tackle initial task tickets within 24–48 hours of acceptance.

Competitive Problem Solving

ALGORITHMS

Strong foundation in algorithms and data structures (LeetCode / HackerRank style problem solving), ensuring rapid resolution of complex logic puzzles and edge cases.

Clear Communication

COLLABORATION

Fluent written English communication for asynchronous GitHub PR discussions, technical RFCs, issue comments, and documentation.

Ready for Technical Screening & Immediate Project Tasks

We welcome any technical evaluation, coding challenge, or repository walk-through. Please review our public GitHub repositories and portfolio, and let us know the next steps for technical screening and onboarding.